

ScholarSetu — Validation Report: Code vs. Architecture

Project: SIH 2026 · PS 26238 · Team ACE
Reviewed against: ARCHITECTURE.md
Date: 2026-09-26

1. Summary

The repository follows the architecture's folder layout (§12), and the design vocabulary appears throughout the code: canonical states, claim types, attestations, verifier plugins, Mitra scopes. Very little of that design actually runs yet.

- **Hardcoded data.** Most endpoints return fixed data for the demo characters Sunita and Rahul, whatever the request says.
- **No database.** Nothing is saved to Postgres. All state lives in Python dictionaries and is lost when the server restarts.
- **No real integrations.** The core never calls the mock government services, adapters, event bus consumers or workflows.
- **Contradictory UI.** Some screens make claims that the code contradicts. A judge who asks one follow-up question could catch these, which is the biggest risk for the demo.

Counts: 7 critical security/privacy issues, 8 critical correctness issues, 4 high front-end and demo-honesty issues, and 5 medium consistency issues.

How this was checked

- The whole backend (`services/core/app`), the mocks, the migration, the rules, the smoke test, the React console and the Flutter app were read end to end.
- The backend was run in-process with FastAPI's `TestClient` , and the identity resolver was called directly. Every "confirmed" finding below was reproduced this way.
- **Not run:** Docker Compose, the Flutter build and the Capacitor/Android build.

2. Architecture coverage

Architecture component	Section	Status	Notes
D1 Scholarship Passport / attestations	§6.4.4	🟡 Partial	Ed25519 signing works. The key is regenerated on every start, and attestations are stored in memory.
D2 Canonical Lifecycle Ledger	§6.2.1, §6.3	🔴 Stub	Events are kept in a Python list; read models are hardcoded; the hash chain fails its own verification.
D3 Verification Mesh	§6.4	🔴 Stub	All 7 plugins return hardcoded results and none calls a mock service. Verification fails <i>open</i> .
D3 Indic Identity Resolver	§6.4.3	🟡 Unsafe	Runs, but auto-verifies different people; no transliteration.

Architecture component	Section	Status	Notes
D4 Eligibility & Pathway Engine	§6.5	● Stub	<code>eval()</code> on rule strings; <code>rules/*.json</code> never loaded; one-scheme check always says "no conflict".
D5 DBT Guardian	§6.6	● Stub	Health check always returns PASS; the status endpoint crashes (HTTP 500).
D6 Offline-first app	§6.1	● UI only	No Drift, SQLCipher or delta sync; "offline" is a toggle that loads mock data.
D6 Family mode	§6.1	● Partial	Works for the one hardcoded household only.
D6 Mitra mode	§6.1	● Unsafe	No student OTP, no scope or time limits.
D6 SMS / IVR	§6.1	● Missing	Not implemented anywhere.
D7 Reach Radar (PPRL)	§6.8	● Stub	The Bloom-filter code exists but is never called; endpoints return fixed numbers.
Consent Manager	§6.9	● Stub	<code>verify_consent()</code> always returns <code>True</code> and nothing calls it.
Nudge & Notification Engine	§6.10	● Stub	<code>handle_event()</code> is <code>pass</code> ; nothing subscribes to events.
Scheme Adapter Layer	§6.2	● Dead code	Nothing imports it; the NOS adapter crashes on import.
JAGO Scholarship Skill	§6.7	● Stub	Tool results are hardcoded and contradict the ledger.
Temporal workflows / SLA timers	§5.1, §6.3	● Missing	Not in <code>docker-compose.yml</code> , not in code.
NATS JetStream	§5.1	● Present, unused	The container runs, but the app always uses <code>InMemoryEventBus</code> .
Keycloak / OIDC	§9	● Missing	Replaced by a homemade JWT with an insecure default secret.
MinIO	§11	● Missing	Only an environment variable; no service.
PostgreSQL persistence	§7	● Broken	The migration can't run, and nothing writes to the DB.

Legend: ● done · ● partial · ● works but unsafe · ● stub or broken · ● missing

3. Critical: Security & Privacy

S1. Anyone can obtain a MINISTRY-role token — *confirmed*

- OTP `123456` is accepted for **every** phone number: `gateway/router.py:75`.
- The caller chooses their own role in the request body: `gateway/router.py:28`.
- Generated OTPs are also returned in the response as `dev_hint`: `gateway/router.py:66`.

```
curl -X POST localhost:8000/v1/auth/otp/verify -H 'Content-Type: application/json' \
  -d '{"phone":"9000000001","otp":"123456","role":"MINISTRY"}'
# -> 200, JWT with "role":"MINISTRY"
```

Fix: remove the universal OTP. Assign roles on the server from a user table, never from the request.

S2. Requests without a token are still let through

- A request with no `Authorization` header is treated as Sunita: `dependencies.py:17`.
- `get_current_officer` upgrades any non-officer to `DISTRICT_OFFICER`: `dependencies.py:50`.

- Most routers never use these dependencies anyway. Every `/v1/me/*` endpoint takes `?student_id=` from the query string, so anyone can read any student's wallet, passport, payments or pending actions (an IDOR — insecure direct object reference).

Fix: require a valid JWT on every non-public route, take `student_id` from the token, and enforce roles with `Depends`.

S3. Consent is never enforced — contradicts §6.9 and the DPDP claims in §9 — *confirmed*

- The verification router builds its own consent artefact from any string the client sends, falling back to `"cst-auto-001"`: `verification/router.py:17`.
- `ConsentService.verify_consent()` always returns `True`; `list_consents()` returns `[]`; `revoke_consent()` does nothing: `consent/service.py:27`.
- `DELETE /v1/consents/does-not-exist` returns `{"status":"revoked"}`.

Fix: store consents in the database and look them up in verification and wallet calls. Reject calls whose consent is missing, expired or revoked.

S4. Mitra (assisted) sessions skip the student's OTP — *confirmed*

`POST /v1/mitra/sessions` immediately creates an `ACTIVE`, `FULL_ACCESS` session for any `student_id` and any duration. A 100,000-minute session was accepted. The design requires student OTP consent, a scope limit, a time box and an audit log: `gateway/router.py:109`.

S5. Verification fails open — *confirmed*

- If every plugin fails, `verify_single_claim()` returns `VERIFIED`, **confidence 0.96**: `verification/service.py:177`.
- `SubjectRef` is hardcoded to Sunita's identity for every `student_id`. Verifying `stu-random-999` produced a `VERIFIED HIGHER_ED` claim built from Sunita's data.

Fix: when no source can confirm a claim, return `MANUAL_REVIEW` or `SOURCE_UNAVAILABLE` and never `VERIFIED`. Load the subject from the student record.

S6. Eligibility rules are executed with `eval()`

`eligibility/service.py:41`. §6.5 plans for policy admins to edit rules. Once they can, this becomes remote code execution.

Fix: evaluate `rules/*.json` with JSON-Logic or GoRules ZEN, as the architecture specifies. The JSON files already use a `field / operator / value` format that suits this.

S7. Secrets and key handling

- `private.pem` is in the repo root and in `services/core/`.
 - There is no `.dockerignore`, so the private key and `services/core/.env` (which has slots for Gemini, OpenAI and Bhashini keys) are copied into the Docker image.
 - `.gitignore` doesn't exclude `*.pem`. The folder isn't a git repo yet, so the next `git add` would commit the key.
 - `JWT_SECRET` defaults to `"secret": config.py`.
 - CORS allows `"*"` together with `allow_credentials=True`: `main.py`.
-

4. Critical: Correctness

C1. The Identity Resolver auto-verifies different people — confirmed

Results from running `IndicIdentityResolver` directly:

Input	Result	Expected
"Sunita Hansda" vs "Anita Hansda" (twin: same DOB, father, district)	AUTO_VERIFY, 1.00	Manual review
"Sunita Hansda" vs "Sunita Murmu" (same DOB, father, district)	AUTO_VERIFY	Manual review
Near-exact name, zero corroborating fields	AUTO_VERIFY, 0.95	§6.4.3 requires ≥ 1 corroborating field
"Srinivas Munda" vs "Nivas Munda"	1.00	Low
"Shrikant Oraon" vs "Kant Oraon"	1.00	Low
"Ravi Kumar Soren" vs "Ravi Soren"	1.00 ("Kumar" deleted)	High, but for the right reason
"Sunita Hansda" vs "सुनीता हांसदा"	0.08	High

Root causes:

- **Additive corroboration.** Corroboration (up to +0.35) is *added* to the name score, so matching family details hide a different first name: `identity_resolver.py:71`.
- **Substring removal.** Honorifics are stripped with `str.replace`, so `sri` inside "Srinivas" and `shri` inside "Shrikant" are deleted, and so is the real name "Kumar": `identity_resolver.py:113`.
- **No transliteration.** `_transliterate()` returns its input unchanged, and token alignment returns `[]`. `indic-transliteration` is already in `requirements.txt` but unused.
- **Accidental demo result.** The demo's "Hansda vs Hansdah → provisional" outcome happens only because the Devanagari comparison fails. Hansda vs Hansdah on its own scores 0.99, which would auto-verify.

Fix:

- remove honorifics by whole token only;
- transliterate every name to Latin script before comparing;
- use corroboration as a **gate** (at least one field must match for auto-verify), not as a bonus;
- compare given names and surnames separately;
- build `tests/identity_matching_eval/` with labelled pairs, including hard negatives such as twins and siblings.

C2. The ledger's hash chain fails its own integrity check — confirmed

The seed events are hashed with payload `"{}"` (`ledger/service.py:81`) but stored with real payloads. `verify_chain("APP-PM-2026-000812")` returns **False**, so the tamper-evidence feature reports the demo data itself as tampered.

C3. Eligibility never returns `eligible: true` — confirmed

The hardcoded `StudentMock` has no `INCOME` attestation, so Pre-Matric, Post-Matric and every other scheme come back ineligible. In addition:

- the student record is ignored (`student_id` is unused);
- `check_one_scheme_rule()` always returns "no conflict", and `POST /v1/applications` never calls it;
- `get_pathway()` is hardcoded;
- the `RuleVersion` table and `rules/*.json` are never used.

C4. Application IDs collide — confirmed

The ID is `APP-PM-2026-{last 4 chars of student_id}` (`ledger/router.py:40`). The same student's NOS and Pre-Matric applications both got `APP-PM-2026-tu-x` and share one hash chain. New applications are also never added to `_applications`, so `transition_state()` can't find them.

C5. JAGO's money answers contradict the ledger — confirmed

This is the exact failure §6.7's "money-safe AI" is meant to prevent.

Source	Sanctioned	Credited	Year
Ledger <code>/v1/me/payments</code>	₹14,500	₹0	2026-27
JAGO <code>_get_payments()</code> (<code>jago_skill/service.py:171</code>)	₹12,000	₹12,000	2023-24

Other JAGO problems:

- **Wrong routing.** "Mera paisa kab aayega?" is routed to a `status` answer about app `APP123`, because the keyword `kab` is checked before `paisa`: `jago_skill/tools.py:23`.
- **Guideline search almost never matches.** A result is returned only when the entire question appears word for word in a guideline, so "What is the income limit for post matric?" gets no answer.
- **No citations.** `citations` is always `[]`.
- **Crashes on other languages.** Any language except `hi/en` (e.g. Santali, `sat`) returns **HTTP 500**.
- **Leaks errors.** Bad tool parameters return 500 with the internal Python error message.
- **Unauthenticated tool route.** `/v1/skill/tools/*` has no mTLS or authentication and takes any `student_id`.

Fix: have JAGO's tools call `LedgerService`, `EligibilityService` and `AttestationService` directly. Test that every rupee figure JAGO outputs matches `/v1/me/payments`.

C6. Endpoints and modules that crash

Location	Problem
<code>GET /v1/dbt/status/{id}</code>	HTTP 500: <code>PaymentState.PENDING</code> doesn't exist in the enum. <code>dbt_guardian/service.py:80</code>
<code>adapters/nos_adapter.py:9</code>	Uses <code>SourceSystem.NOS</code> , but the enum is <code>NOS_PORTAL</code> , so importing the file raises <code>AttributeError</code> .
All three adapters	<code>Path(__file__).parents[4]</code> raises <code>IndexError</code> in Docker (the path is <code>/app/app/...</code>), and <code>adapters/*.yaml</code> isn't in the Docker build context.
<code>state_mapping.py:26</code>	Unknown source states silently become <code>DRAFT</code> . §6.2.2 requires <code>park_and_alert</code> .

C7. The database layer cannot work

- `alembic/env.py` imports no models, so autogenerate sees nothing.

- `ledger/models.py` and `gateway/models.py` are placeholder tables (`ledger_table` , `gateway_table`).
- **The migration's enums don't match the Python enums:**

Enum	Migration	Code
<code>scheme_type_enum</code>	HIGHER_EDUCATION , FELLOWSHIP , NATIONAL_OVERSEAS	TOP_CLASS , NFST , NOS
<code>canonical_state_enum</code>	VERIFIED_INSTITUTE , DISBURSED , DEFICIENT , ...	INSTITUTE_VERIFICATION , CREDITED , DEFICIENCY_RAISED , ...
<code>claim_type_enum</code>	CASTE , ACADEMIC , BANK	ST_STATUS , ACADEMIC_RECORDS , ... (11 values)
<code>attestation_statuses_enum</code>	no PROVISIONAL	has PROVISIONAL

- `CREATE EXTENSION vector` (`001_initial.py:35`) fails on the `postgres:16` image. Use `pgvector/pgvector:pg16` .
- **No service reads from or writes to Postgres.**

C8. Attestations can't be verified after a restart

- A new Ed25519 key is generated on every start (`attestation/service.py:17`), and `ATTESTATION_PRIVATE_KEY_PATH` is never read. Every attestation issued before a restart becomes unverifiable.
- The signature leaves out `method` , `confidence` , `evidence_hash` and `status` , so those fields can be changed without detection.
- The format isn't JWS or a W3C Verifiable Credential as §6.4.4 states, and no endpoint publishes the public key.
- Attestation IDs are unique only to the second.

5. High: Front-ends & Demo Honesty

F1. The officer/ministry console never calls the backend

- The base URL is `${host}/api` (`client.ts:18`), but the backend serves `/v1/...` .
- The `useApi` hook is **never used**. Every page renders hardcoded arrays; the only live call is JAGO chat in `StudentPortal.tsx` .
- Approving a review case changes only local React state, yet the UI says *"Audit record committed to ledger"*: `ReviewQueue.tsx:107` .
- The review screen shows *"94.2% (Double Metaphone Match)"* and says the Devanagari name *"matches 100%"* (`ReviewQueue.tsx:43`). No Double Metaphone exists in the code, and the real Devanagari score is 0.08.

F2. The Flutter app claims more than it does

- Its only dependencies are `http` , `intl` , `provider` and `shared_preferences` : no Drift, SQLCipher, FCM or Riverpod as §11 lists.
- The "offline outbox" is an in-memory list that is lost when the app is closed.
- The offline toggle says *"Using local SQLCipher encrypted store"* (`dashboard_screen.dart:106`), which is untrue.
- Every API failure silently falls back to built-in mock data, so a broken backend looks healthy.

- The Mitra screen has the OTP pre-filled as 123456 and never contacts the backend.

F3. The README contradicts the architecture and the code

README claim	Reality
Mobile app is "Capacitor Android"	The architecture says Flutter; both now exist (apps/mobile and apps/console/android). Pick one.
Mocks at :8100/aadhaar , :8100/npci , plus a Bhashini mock	Actual paths are /uidai and /pfms/npci ; there is no Bhashini mock.
"Verifies the signature in 5ms"	Not measured anywhere.
"Offline-verifiable passport with QR code"	No QR generation exists.
"4 million students", "over 30% fall through"	No source is given.

F4. The smoke test only proves HTTP 200

scripts/demo_smoke_test.py checks status codes, never content:

- Scene 4 expects a seeding failure but gets PASS , and the test still prints "ALL 8 SCENES PASSED CLEANLY".
- Scene 6 (offline + SMS) is skipped.
- There are no unit tests anywhere, and the tests/ tree from §12 doesn't exist.

6. Medium: Consistency

1. **Scheme parameters disagree across three sources.** - Top Class income limit: ₹8,00,000 in decision_tables.py:75 , but ₹6,00,000 in the JAGO RAG text. - Pre-Matric school enrolment: required in the Python rules, absent from rules/pre_matric_2026_v1.json . - Verify every limit and amount against the current MoTA guidelines, and keep them in one place (rules/*.json) with a citation. For example, the NFST JRF rate of ₹31,000/month should be checked against the latest UGC revision.
2. **Mock data contradicts the demo story.** - The e-District verifier returns tribe "Gond", while Sunita is Santal everywhere else. - The DigiLocker "ST status" verifier returns an income figure. - The PFMS/NPCI mock always returns seeded: true , so even a real DBT Guardian could never show the Scene 4 failure.
3. **PPRL privacy and matching gaps.** - The HMAC key is hardcoded (b"scholarsetu-pprl-key"). - The Bloom filter has no salting and no protection against frequency attacks. - Normalisation strips every non- a-z character, so Devanagari names encode to empty. - One enrolment record can match several scholarship records (no 1:1 matching).
4. **The review pipeline isn't wired up.** - A PROVISIONAL result never creates a review case or a provisional attestation (§6.4.1). Only the one seeded case exists. - Officer decisions accept any VerificationStatus (e.g. SOURCE_UNAVAILABLE) instead of ReviewDecision . - Decisions don't write a ledger event.
5. **Events go nowhere.** Each service builds its own in-memory event bus and nothing subscribes, so the "event → read model → notification" flow of §5.3 never happens.

7. Remediation Plan

Ordered to match the "if time is short" priority in ARCHITECTURE.md §15, with security first.

#	Work item	Fixes	Effort
1	Security floor. Remove the universal OTP, role self-selection and the no-token fallback. Take <code>student_id</code> from the JWT. Add a real Mitra OTP step, <code>.dockerignore</code> , <code>*.pem</code> in <code>.gitignore</code> , key loading from file, and strong secrets.	S1, S2, S4, S7, C8	S
2	Verification fails closed. Return <code>MANUAL_REVIEW</code> when every source fails. Load the subject per student. Create review cases and provisional attestations from real results.	S5, M4	S
3	Identity Resolver. Whole-token honorific removal, transliteration, corroboration as a gate, a labelled evaluation set with hard negatives.	C1	M
4	Persistent ledger. Fix the migration enums and pgvector image; implement real <code>ledger_events / applications</code> models; compute hashes consistently; derive all read models from events; unique application IDs.	C2, C4, C7	M
5	One source of truth for JAGO. Tools call ledger, eligibility and attestation services; payment intent checked before status; real guideline retrieval with citations; language fallback instead of 500.	C5	S
6	Rules-as-code. Replace <code>eval()</code> with JSON-Logic over <code>rules/*.json</code> ; implement the one-scheme check against the ledger; record <code>rule_version</code> on each decision.	S6, C3, M1	M
7	Connect the core to the mocks. Plugins, adapters and DBT Guardian call <code>MOCK_SERVICE_URL</code> . Seed the mocks with the demo scenarios, including the unseeded account and the Hansdah spelling. Fix the adapter import and path bugs; implement <code>park_and_alert</code> .	C6, M2	M
8	Consent Manager. Persist consents; enforce them in the verification and wallet paths; make revocation real.	S3	S
9	Console. Point it at <code>/v1</code> , use <code>useApi</code> , and send review decisions to the backend. Remove the Double Metaphone and "committed to ledger" text unless it becomes true.	F1	S
10	Mobile. Choose Flutter or Capacitor. If Flutter: add Drift + SQLCipher and a persistent outbox, and stop silently falling back to mock data (show "last updated at" instead).	F2	M-L
11	Tests. Rewrite the smoke test to check content (DBT shows the seeding issue; JAGO's amount matches the ledger; <code>verify_chain</code> is true). Add unit tests for the resolver, the hash chain and the rules.	F4	S
12	Docs. Bring the README in line with the code and the architecture; source or remove the statistics.	F3	S

Effort: **S** ≈ under half a day · **M** ≈ 1–2 days · **L** ≈ 3+ days.

Before any live demo

At a minimum, fix **S1, S5, C1, C2, C5 and F1**. These are the issues a judge is most likely to hit by asking one follow-up question or trying one extra input:

- "log in as the Ministry";
- "try a sibling's name";
- "verify the audit chain";
- "ask JAGO about money";
- "refresh the console after approving".